



Escuela Técnica Superior de
Ingeniería Informática

TRABAJO FIN DE GRADO

Desarrollo de un Agente Conversacional basado en IA para la Gestión Burocrática en la Universidad de Sevilla

Realizado por
Kevin Amador Calzadilla

Para la obtención del título de
Grado en Ingeniería Informática - Ingeniería del Software

Dirigido por
José Enrique Sánchez López y Pablo Reina Jiménez

En el departamento de
Lenguajes y Sistemas Informáticos

15 de abril de 2026

Índice general

I	Introducción y Conceptos	1
1.	Introducción	2
1.1.	Marco conceptual	2
1.2.	Motivación	3
1.3.	Objetivos	3
1.3.1.	Objetivo general	4
1.3.2.	Objetivos técnicos	4
2.	Preparación del proyecto	5
2.1.	Marco Tecnológico y Herramientas utilizadas	5
2.2.	Lenguaje de programación	5
2.3.	Orquestación del agente y Ecosistema RAG	6
2.4.	Modelos de Inferencia y Procesamiento	6
2.5.	Procesamiento de Documentos Burocráticos	6
2.6.	Desarrollo de Backend e Interfaz de Usuario	6
2.7.	Herramientas de Evaluación	6
2.7.1.	Comparativa de Rendimiento: Entorno Local vs Groq	6
3.	Conceptos iniciales	8
3.1.	Agentes de IA frente a LLMs	8
II	Metodología y planificación	9
4.	Metodología	10
4.1.	Ciclo de Vida del Proyecto	10
4.2.	Metodología Scrum en la ejecución	10
III	Ejecución	11
5.	Fase 1: Ingesta y Optimización del Conocimiento	12
5.1.	Análisis de Chunks y Contexto	12

5.2. Optimización: El problema del doble splitter	12
6. Fase 2: Diseño del Agente y Dual RAG	13
6.1. Arquitectura del Router y Nodos de Decisión	13
6.2. Implementación del Dual RAG	13
7. Fase 3: Optimización de Rendimiento	14
7.1. Migración de Infraestructura: De HuggingFace a Groq	14
8. Fase 4: Evaluación y Validación	15
8.1. Framework RAGAS	15
8.2. Batería de validación	15
IV Resultados y Conclusiones	16
9. Resultados	17
9.1. Análisis de precisión	17
10. Conclusiones	18
10.1. Lecciones aprendidas	18
10.2. Trabajo futuro	18
A. Uso de la Inteligencia Artificial	20

Parte I

Introducción y Conceptos

Capítulo 1

Introducción

1.1. Marco conceptual

El avance de los Modelos de Lenguaje Grande (LLMs)

Los modelos de Lenguaje Grande (LLMs) han progresado de manera muy veloz en los últimos años, dejando muy en claro su alta capacidad para entender y generar lenguaje natural. Sin embargo, cuando se trata de buscar la respuesta más precisa a un usuario el uso del modelo únicamente no permite ofrecerle la respuesta más óptima. Siendo así, que en ocasiones estos llegan a alucinar, generando respuestas que suenan convincentes pero que carecen de veracidad o no tienen relación exacta con lo que solicita el usuario.

De modelos de lenguaje a estructuras agénticas

Para superar esta restricción técnica, se propone como solución el uso de arquitecturas basadas en agentes de Inteligencia Artificial. Las estructuras agénticas proporcionan al sistema habilidades para poder razonar en función de lo que les ofrece el usuario. Gracias a esto, pueden llevar a cabo un proceso lógico de toma de decisiones, evaluando el contexto y utilizando herramientas externas si lo necesitan. Debido al razonamiento que realizan, pueden ofrecer una respuesta mucho más óptima al usuario.

Inyección de contexto: Embeddings y arquitectura RAG

Atendiendo al escenario específico de este trabajo, debemos sumarle a esa capacidad de razonamiento la necesidad de operar únicamente sobre información veraz y oficial. Para resolver este aspecto, se integran arquitecturas de Generación Aumentada por Recuperación (RAG) apoyadas en bases de datos vectoriales. Mediante el uso de modelos de *embeddings*, los documentos se transforman en vectores, lo que más adelante permite al sistema realizar búsquedas semánticas. De este modo, se recupera únicamente el contexto que el modelo necesita, evitando alucinaciones y asegurando respuestas que se basen únicamente en los documentos ofrecidos.

1.2. Motivación

En ocasiones, la gestión administrativa en una institución tan grande como lo es la Universidad de Sevilla conlleva, casi de forma inevitable, a una gran cantidad de documentación dispersa. A día de hoy, cualquier estudiante, profesor, administrativo o aspirante a una plaza docente se enfrenta a un auténtico caos de normativas de matriculación, complejos de baremos de contratación, calendario académicos, guías de evaluación, etc. A pesar de que toda esta información es pública, pero su consulta se suele hacer altamente compleja.

La realidad ante esta situación es que cualquiera relacionado con este entorno se ve obligado a navegar durante horas en archivos PDF con decenas de páginas para simplemente poder localizar un dato específico. Esto no solo genera frustración y pérdida de tiempo, sino que el lenguaje administrativo, a menudo denso y técnico, propone una barrera que dificulta la interpretación correcta de los procesos.

Este problema termina haciendo que en ocasiones se tenga que consultar más veces de las requeridas las secretarías de los centros. Por lo que, el personal de administración y servicios (PAS) se ve obligado a invertir gran parte de su jornada en responder a correos y llamadas con dudas repetitivas. En su gran mayoría, todas estas consultas ya se han resuelto previamente en los documentos oficiales, pero debido a la extensión de los mismos y a la gran cantidad que se presentan los usuarios no son capaces de encontrarlos fácilmente.

Es en este escenario donde este Trabajo de Fin de Grado cobra sentido. La necesidad de un asistente que se encuentre operativo las 24 horas del día facilitaría mucho a aquellos usuarios que necesitasen resolver su duda al instante sin ningún tipo de espera. El valor de esta propuesta no reside únicamente en buscar palabras clave, sino en la capacidad de entender el lenguaje natural con todo el contexto que nos ofrece el usuario, sintetizar la información y explicarla de la manera más clara y detallada al usuario. Al integrar un agente que razona y cita la fuente exacta de la normativa, transformamos un proceso burocrático poco intuitivo, por su complejidad a la hora de encontrar la información, en una interacción directa, fiable y eficiente, haciendo tanto a los usuarios como a los administradores de la institución ahorrar tiempo.

1.3. Objetivos

El propósito fundamental de este proyecto es transformar la interacción entre el usuario y la densa normativa de la Universidad de Sevilla mediante el uso de tecnologías de vanguardia en el campo de la Inteligencia Artificial. A continuación, se detallan las metas perseguidas tanto a nivel global como técnico.

1.3.1. Objetivo general

El objetivo principal de este trabajo consiste en diseñar, desarrollar y validar un asistente conversacional inteligente basado en arquitecturas RAG (*Retrieval-Augmented Generation*) que sea capaz de asistir únicamente apoyándose en los documentos facilitados por los administradores a candidatos, estudiantes, docentes. El sistema debe ser capaz de interpretar cualquier tipo de consulta compleja sobre trámites burocráticos y normativos de la Universidad de Sevilla, en caso de que se encuentre en los documentos, ofreciendo de esta forma respuestas precisas para aclarar al máximo la cuestión del usuario.

1.3.2. Objetivos técnicos

Para alcanzar el objetivo general, se han definido una serie de hitos que abordan los principales retos identificados durante el desarrollo:

- **Optimización de la Ingesta Semántica:** Desarrollar una forma de procesar datos capaz de interpretar documentos con estructuras complejas. Esto implica el uso de herramientas de análisis como es Docling, para tras ello realizar una fragmentación (*chunking*) inteligente, garantizando que la información en tablas e imágenes no pierda sentido al ser vectorizada.
- **Arquitectura de Enrutamiento Inteligente:** Implementar un sistema de lógica agéntica (*Routing*), el cual se encargará de actuar como clasificador de intenciones en tiempo real. Este componente se encarga de dirigir cada consulta hacia el nodo especializado correspondiente: los nodos resultadores para respuestas directas, el nodo consultor que preguntará para resolución de ambigüedades o el nodo de rechazo para gestionar las consultas que no tengan ningún tipo de relación con el contexto del agente.
- **Diseño de un Sistema Dual RAG:** Construir una arquitectura de información que permita dinámicamente separar el conocimiento normativo de las directrices de estilo y buenas prácticas de atención al usuario, permitiendo así que la respuesta sea técnicamente correcta pero sin perder la cercanía hacia el usuario final.
- **Minimización de la Latencia de Respuesta:** Mejorar el desempeño del sistema con el fin de asegurar una experiencia fluida para el usuario. La meta es disminuir los tiempos de respuesta del agente a menos de 20 segundos, empleando modelos de lenguaje livianos pero eficaces para decisiones que no son tan relevantes, como la respuesta final al usuario, y combinando APIs con un alto rendimiento como Groq.

Capítulo 2

Preparación del proyecto

En el desarrollo de un agente conversacional basado en Inteligencia Artificial, la elección del ecosistema tecnológico es un factor crítico que determina no solo el rendimiento del sistema, sino también su escalabilidad y mantenibilidad. En este capítulo se detallan y justifican las herramientas, lenguajes y plataformas empleadas para dar solución a los objetivos planteados.

2.1. Marco Tecnológico y Herramientas utilizadas

Se ha empleado Python como lenguaje principal, utilizando el ecosistema de LangChain para la orquestación del agente y Groq como proveedor de inferencia LLM para garantizar respuestas en tiempo real.

2.2. Lenguaje de programación

Debido a que es un proyecto que requiere de una gran cantidad de librerías y herramientas relacionadas con la inteligencia artificial, se ha optado por el uso de Python como lenguaje principal, dado que es el lenguaje más utilizado en este campo.

2.3. Orquestación del agente y Ecosistema RAG

2.4. Modelos de Inferencia y Procesamiento

2.5. Procesamiento de Documentos Burocráticos

2.6. Desarrollo de Backend e Interfaz de Usuario

2.7. Herramientas de Evaluación

2.7.1. Comparativa de Rendimiento: Entorno Local vs Groq

A lo largo del desarrollo de la validación técnica, se evaluaron dos enfoques arquitectónicos diferentes para la inferencia de modelos en el agente LangGraph: la ejecución local de Llama 3.1 8B y la ejecución a través de la infraestructura cloud de Groq (usando Llama-3-70B).

- **Precisión y Comprensión (8B vs 70B):** La versión local de 8B parámetros probó ser robusta siguiendo instrucciones en fases iniciales, evitando errores de enrutamiento al actuar como *clasificador*. Sin embargo, al enfrentar tareas que exigen razonamiento deductivo complejo, como interpretar la casuística de artículos en el Baremo de Contratación, la API de Groq con el modelo de 70B parámetros demostró una capacidad cognitiva y de resumen inmensamente superior.
- **Latencia y LLM TTFT (Time To First Token):** La diferencia más crítica fue el rendimiento en tiempo real. En el entorno local, los tiempos de inferencia introducían una penalización de varios segundos (comúnmente >10s) en cada subnodo del grafo del agente. Gracias al hardware propietario de inferencia especializado (LPUs) de Groq, estos bloqueos se redujeron a fracciones de segundo por ejecución completa, trasladando el tiempo de espera casi íntegramente a las consultas del Pipeline RAG (búsqueda en Pinecone).
- **Gestión de flujos Multi-Turno:** Empíricamente, la migración a un modelo cognitivamente más grande facilitó la creación de flujos conversacionales compuestos. Mientras que Llama Local finalizaba su cadena abruptamente, la implementación paralela sobre Groq habilitó interacciones continuas —donde el agente despliega resúmenes del *contexto general recuperado antes* de interrogar activamente al usuario (modo 'Entrevistador')— logrando un caso de uso asistencial mucho más fiel a las especificaciones.

El contraste entre las limitaciones de una arquitectura perimetral (local) frente a la nube evidencia cómo dicho salto escala drásticamente el valor de este sistema al transformarlo en una solución viable en productivo.

Capítulo 3

Conceptos iniciales

3.1. Agentes de IA frente a LLMs

Mientras que un LLM es un modelo estadístico de predicción de texto, un agente de IA utiliza razonamiento y herramientas externas para ejecutar tareas, gestionar memoria y tomar decisiones basadas en el contexto del usuario.

Parte II

Metodología y planificación

Capítulo 4

Metodología

4.1. Ciclo de Vida del Proyecto

El proyecto se ha desarrollado siguiendo un enfoque iterativo, permitiendo el ajuste de los modelos de embedding y las estrategias de fragmentación según los resultados de las pruebas.

4.2. Metodología Scrum en la ejecución

Se han definido Sprints centrados en hitos técnicos: ingesta, diseño de nodos, optimización y evaluación final.

Parte III

Ejecución

Capítulo 5

Fase 1: Ingesta y Optimización del Conocimiento

5.1. Análisis de Chunks y Contexto

La fragmentación o chunking es crítica para que el modelo reciba la información pertinente sin exceder su ventana de contexto.

5.2. Optimización: El problema del doble splitter

Durante el desarrollo se detectó una degradación en la calidad de las respuestas, especialmente al consultar tablas de baremos. El análisis reveló que se estaban ejecutando dos splitters de forma redundante: el de Docling y el de LangChain. Al eliminar el splitter de LangChain y confiar exclusivamente en la segmentación semántica de Docling, se preservó la estructura de las tablas y mejoró drásticamente el contexto inyectado al LLM.

Capítulo 6

Fase 2: Diseño del Agente y Dual RAG

6.1. Arquitectura del Router y Nodos de Decisión

El sistema emplea un nodo Router inicial que clasifica la intención del usuario. Dependiendo de la consulta, el flujo se deriva a nodos específicos: Rechazo (fuera de dominio), Baremo, Calendario, Normativo o Procedimental.

6.2. Implementación del Dual RAG

Se diseñó un sistema de dos RAGs paralelos. Uno especializado en "Buenas Prácticas" de atención al usuario y otro enfocado estrictamente en la normativa técnica y ficheros de entrenamiento de la US.

Capítulo 7

Fase 3: Optimización de Rendimiento

7.1. Migración de Infraestructura: De HuggingFace a Groq

Inicialmente, el uso de modelos en HuggingFace generaba latencias de hasta 15 segundos. Dado que el objetivo es un asistente en tiempo real, se migró a la API de Groq. Esta transición permitió reducir los tiempos de respuesta a una media de 5 segundos, mejorando significativamente la usabilidad del agente.

Capítulo 8

Fase 4: Evaluación y Validación

8.1. Framework RAGAS

La efectividad del sistema se evaluó mediante RAGAS, comparando la respuesta generada y el contexto recuperado contra un Ground Truth definido. Se midieron parámetros como la fidelidad y la relevancia de la respuesta.

8.2. Batería de validación

Se procesó una batería de 50 preguntas diseñadas para estresar cada nodo del agente, validando tanto la clasificación del router como la precisión de la recuperación de datos.

Parte IV

Resultados y Conclusiones

Capítulo 9

Resultados

9.1. Análisis de precisión

El sistema demuestra una alta fiabilidad en la clasificación de consultas. La optimización del splitter permitió que el 90 % de las consultas sobre baremos devolvieran datos exactos.

Capítulo 10

Conclusiones

10.1. Lecciones aprendidas

La importancia de una ingesta limpia y la reducción de la latencia son factores determinantes en el éxito de un agente conversacional basado en RAG.

10.2. Trabajo futuro

Como líneas futuras se plantea el despliegue escalable en PostgreSQL y la integración con interfaces de mensajería instantánea como Telegram.

Bibliografía

- [1] Amazon Web Services. ¿Qué es un modelo de lenguaje grande (LLM)? Disponible en: <https://aws.amazon.com/es/what-is/large-language-model/>
- [2] LangChain AI. LangChain Documentation. Disponible en: <https://python.langchain.com/>
- [3] RAGAS Framework. Automated Evaluation of RAG. Disponible en: <https://docs.ragas.io/>

Apéndice A

Uso de la Inteligencia Artificial

En cumplimiento con las directrices académicas, se declara el uso de modelos de lenguaje para la asistencia en la generación de código LaTeX y el refactor de scripts de evaluación.